

Multi-Agent - RAG - LLM-Powered

AI Note Document

A complete technical note covering the architecture, AI agent design, system prompts, multi-agent workflows, API reference, and deployment guide for the Instaboot intelligent developer assistant.

React.js Flask Python Groq LLM ChromaDB sentence-transformers SQLite

Docker Compose

Prepared by Vikram Saravanan - Instaboot Project

CONFIDENTIAL - AI NOTE - 2025

INTRODUCTION

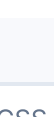
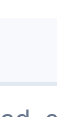
What is Instaboot?

Instaboot is a full-stack, multi-agent AI chatbot designed specifically for developers. It eliminates two of the most common friction points in developer workflows — **getting software installed** and **querying knowledge from documents** — through a purpose-built pipeline of cooperating AI agents.

Unlike a generic chatbot, every message in Instaboot is classified by an Intent Agent before any generation occurs. The right specialist agent then handles the request — producing installation scripts, answering document queries via Retrieval-Augmented Generation (RAG), or engaging in contextual conversation.

CORE CAPABILITIES

Three AI Superpowers

 Intent-Aware Routing Every message is classified before processing. Zero wasted tokens on the wrong agent.	 Script Generator Detects OS + distro, fetches latest version, outputs copy-paste install scripts for winget / brew / apt / dnf.	 RAG Knowledge Base Upload CSV / JSON / text files. Ask questions. ChromaDB retrieves semantically relevant chunks. Groq LLM answers — grounded, no hallucination.
--	---	---

TECHNOLOGY STACK

Stack at a Glance

LAYER	TECHNOLOGY	ROLE
Frontend	React.js · Tailwind CSS · Axios	Chat UI, file upload, script modal, session sidebar
Backend API	Flask · Python 3.10+	REST routes, agent orchestration, CORS handling
LLM	Groq (openai/gpt-oss-120b)	Script generation, knowledge answers, general chat
Embeddings	sentence-transformers/all-MiniLM-L6-v2	384-dim semantic vectors for document chunks
Vector DB	ChromaDB	Persisted vector store, cosine similarity retrieval
Relational DB	SQLite	Sessions, chat history, document metadata
Infra	Docker Compose	Single-command full-stack startup

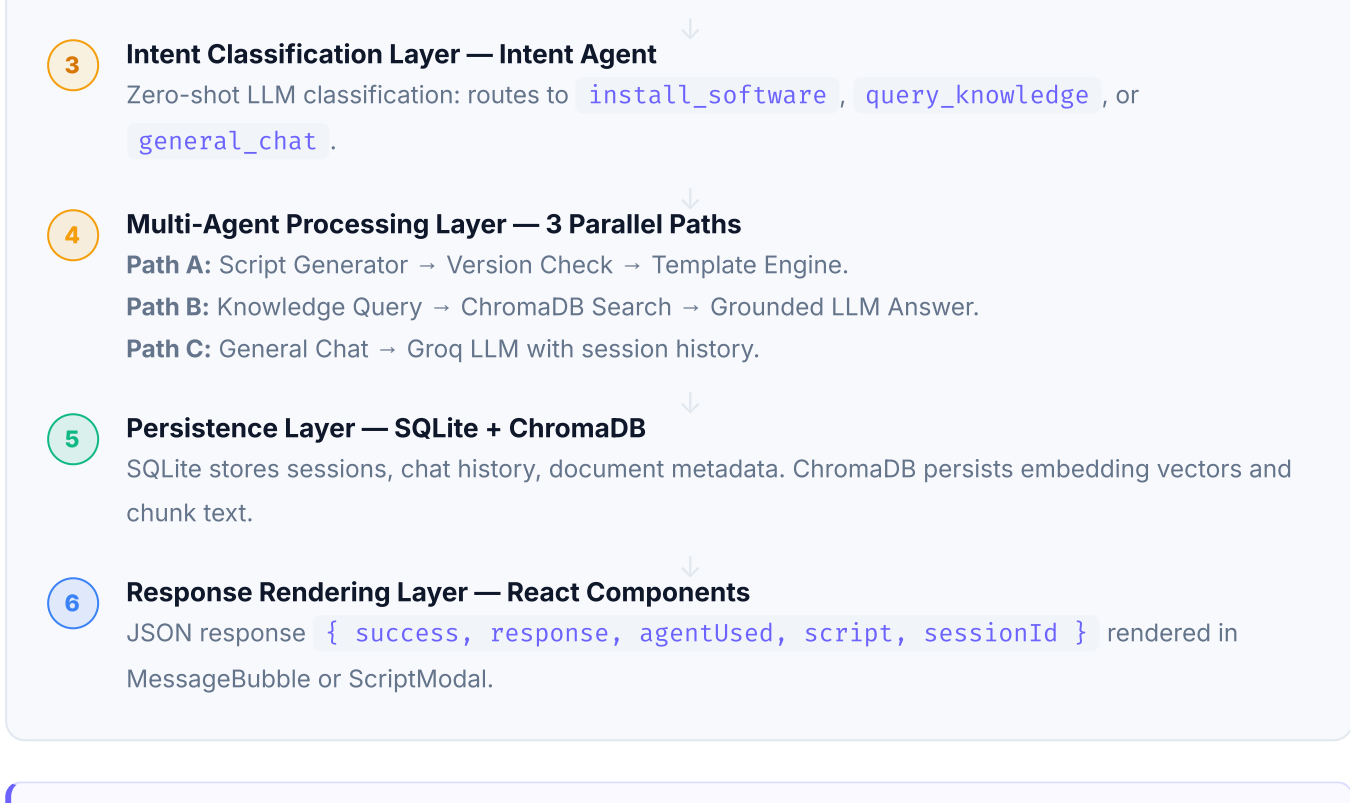
Instaboot — AI Note Document

Vikram Saravanan - 2025

ARCHITECTURE

End-to-End System Design

Instaboot follows a **classify — route — generate — persist — respond** pattern across six distinct layers. No layer is aware of the internals of another, making each independently testable and replaceable.



LLM Call Count per Request Type:
Install Script = **3 LLM calls** (Intent → Entity Extraction → Script Generation) · RAG Query = **1 LLM call** (retrieval is vector math, not LLM) · General Chat = **1 LLM call**

Instaboot — AI Note Document

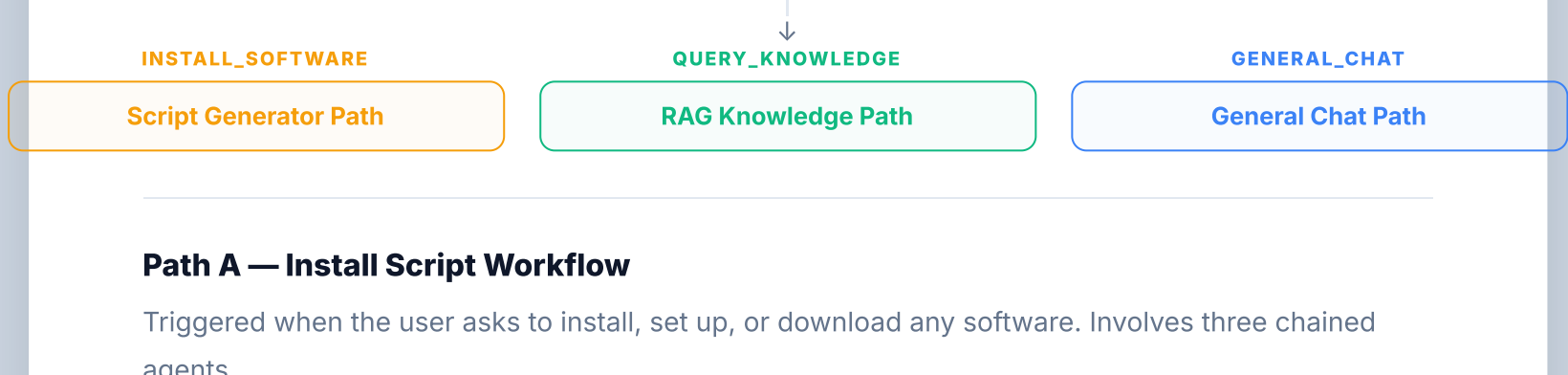
Vikram Saravanan - 2025

MULTI-AGENT WORKFLOW

How Every Message is Processed

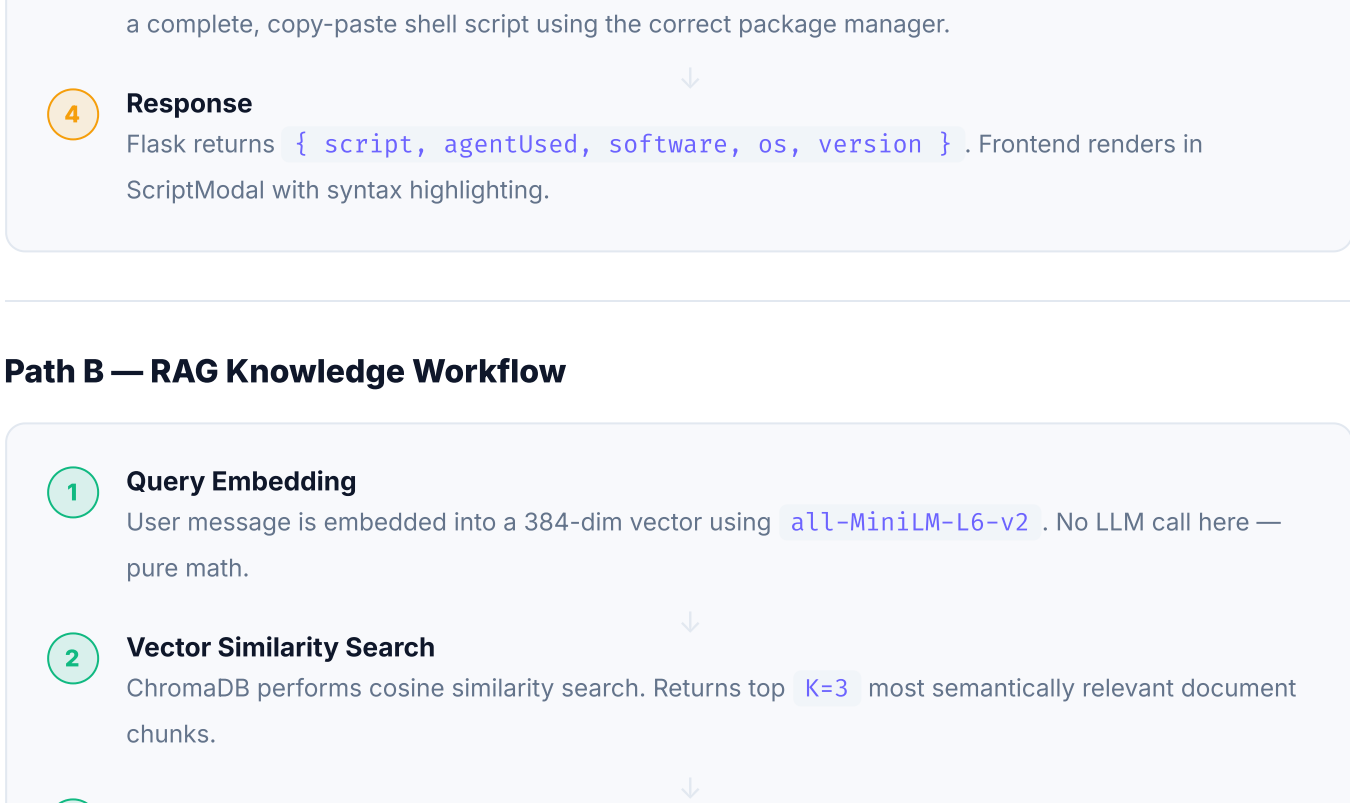
Step 1 — Intent Classification

Every inbound message hits the Intent Agent first. This is a single, fast LLM call that outputs one of three labels.



Path A — Install Script Workflow

Triggered when the user asks to install, set up, or download any software. Involves three chained agents.



Path B — RAG Knowledge Workflow



Instaboot — AI Note Document

Vikram Saravanan - 2025

AI PROMPTS

Agent Prompt Reference

① Intent Agent Prompt

Zero-shot classifier. Outputs a single label with no prose. Runs on every message.

SYSTEM:
You are an intent classification engine for a developer assistant chatbot. Return **ONLY** one of these three labels — nothing else:

`"install_software"` → user wants to install, set up, download, or configure any software
`"query_knowledge"` → user wants to search, query, or retrieve info from uploaded documents
`"general_chat"` → everything else

No punctuation. No explanation. One label only.

USER: {user_message}

② Entity Extraction Prompt (Stage 1 of Script Generator)

Extracts structured fields from natural language. Output is always JSON.

SYSTEM:
You are an entity extractor. Extract the following fields from the user message and return **ONLY** valid JSON. No markdown. No explanation.

Fields:

- `"software"`: name of software (lowercase, e.g. `"docker"`, `"nodejs"`)
- `"os"`: one of `"windows"` | `"macos"` | `"linux"` | `"unknown"`
- `"distro"`: if linux → `"ubuntu"` | `"debian"` | `"fedora"` | `"unknown"`

USER: {user_message}

③ Script Generation Prompt (Stage 2 of Script Generator)

SYSTEM:
You are an expert DevOps engineer. Generate a complete install script.

Rules:

- Use correct package manager → `winget` (Windows) | `brew` (macOS) | `apt` (Ubuntu) | `dnf` (Fedora)
- Include version verification command after install
- Include a smoke-test command where applicable
- Add brief inline `# comments` for non-obvious steps
- Output **ONLY** the script block — no prose outside it

Software : {software} OS : {os} Distro: {distro} Version: {version}

④ Knowledge Query (RAG) Prompt

SYSTEM:
You are a document-grounded QA assistant. Answer the user's question using **ONLY** the context chunks below.

Rules:

- Use **ONLY** the provided context. No outside knowledge.
- Cite the source document name for each fact.
- If context is insufficient, respond: `"I couldn't find relevant information in the uploaded documents."`
- Never fabricate data, numbers, or names.

CONTEXT:

```
--- Chunk 1 (source: {doc_name_1}) ---
{chunk_1}
--- Chunk 2 (source: {doc_name_2}) ---
{chunk_2}
--- Chunk 3 (source: {doc_name_3}) ---
{chunk_3}
```

USER: {user_message}

⑤ General Chat Agent Prompt

SYSTEM:
You are Instaboot, a friendly and knowledgeable developer assistant. Specialise in software development, DevOps, and system administration.

- Use markdown formatting. Specify language in all code fences.
- Maintain continuity using the session history below.
- Never reveal the underlying model or provider. You are "Instaboot".

CONVERSATION HISTORY: {session_history}

USER: {user_message}

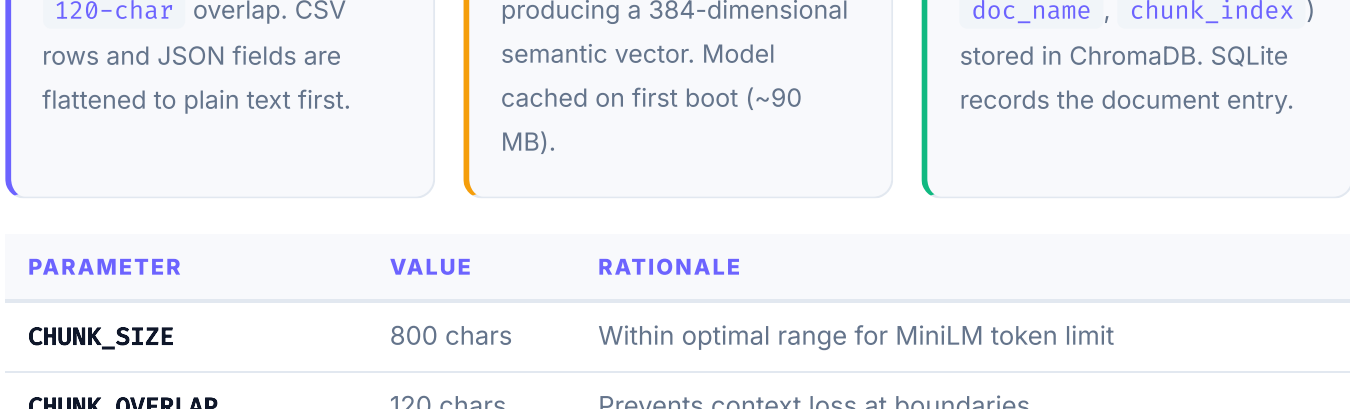
Instaboot — AI Note Document

Vikram Saravanan - 2025

DOCUMENT INGESTION

RAG Ingestion Pipeline

Before any document is queryable, it passes through a three-stage deterministic pipeline. No LLM is involved — this is pure vector math.



PARAMETER	VALUE	RATIONALE
CHUNK_SIZE	800 chars	Within optimal range for MiniLM token limit
CHUNK_OVERLAP	120 chars	Prevents context loss at boundaries
Embedding Dim	384	Fixed output of all-MiniLM-L6-v2
Similarity Metric	Cosine	Standard for semantic text search
TOP_K	3	Enough context without overloading LLM prompt

API REFERENCE

REST Endpoints

METHOD	ROUTE	DESCRIPTION
<code>GET</code>	<code>/api/health</code>	Liveness probe. Returns <code>{ status: "ok", timestamp }</code> . Use for Docker health checks.
<code>POST</code>	<code>/api/upload</code>	Upload CSV or JSON file. Triggers full ingestion pipeline. Returns <code>documentId</code> + <code>chunksCreated</code> .
<code>POST</code>	<code>/api/upload/text</code>	Ingest raw plain text directly. Useful for notes or documentation snippets without a file.
<code>GET</code>	<code>/api/documents</code>	List all indexed documents: id, name, type, created_at.
<code>POST</code>	<code>/api/chat</code>	Core AI endpoint. Routes through full agent pipeline. Returns response, agentUsed, script (if any), sessionId.
<code>GET</code>	<code>/api/chat/history/:id</code>	Full message history for a session, including which agent handled each assistant turn.
<code>GET</code>	<code>/api/chat/sessions</code>	All sessions with message count, timestamps, and first-message preview.

Instaboot — AI Note Document

Vikram Saravanan - 2025

GETTING STARTED

Quick Setup

```
# 1. Clone the repository
git clone https://github.com/VikramSaravanan/Instaboot.git
cd Instaboot

# 2. Configure environment
cp .env.example .env
# Open .env and set GROQ_API_KEY=gsk_...

# 3. Build and launch (Docker Compose — recommended)
docker-compose up --build

# Frontend → http://localhost:3000
# Backend → http://localhost:5000/api/health
```

First Boot Note: The first startup downloads the embedding model (~90 MB). All subsequent starts use the cached model volume. No re-download needed.

Key Environment Variables

VARIABLE	DEFAULT	PURPOSE
GROQ_API_KEY	—	Required. Enables live LLM responses via Groq.
GROQ_MODEL	openai/gpt-oss-120b	The Groq chat model to use for generation.
EMBEDDING_MODEL	all-MiniLM-L6-v2	Sentence-transformer for document embeddings.
TOP_K	3	Chunks injected into LLM context per query.
CHUNK_SIZE	800	Characters per chunk during ingestion.
CHUNK_OVERLAP	120	Overlap between adjacent chunks.
CORS_ORIGIN	http://localhost:3000	Allowed frontend origin.

DESIGN PRINCIPLES

AI Prompt Design Rules

① Single Responsibility Each prompt does exactly one thing: Intent classifies, Script Generator generates. Never mixed.	② Strict Output Contracts Prompts that feed other agents output structured data only — single label, JSON, or version string. No prose allowed.	③ Grounding Over Generation RAG agent is instructed to answer only from retrieved context. Hallucination prevented by explicit prompt rule, not luck.
④ Graceful Fallbacks Every agent has a defined fallback state. Version Check returns <code>"latest"</code> . RAG returns a "not found" message. No undefined pipeline state.	⑤ Minimal Context Injection Only agent receives only the context it needs. Script Generator doesn't see session history. Intent Agent doesn't see document chunks.	⑥ Persona Consistency General Chat Agent is locked to "Instaboot" identity. No underlying model names or provider details ever surface to users.

Instaboot — AI Note Document

Vikram Saravanan - 2025